Training Native Recursive Language Models

Omar Abul-Hassan [1] **Miguel Villanueva** [1] **Josh Bowden** [1]

Abstract

Recursive Language Models (RLMs) extend language models to handle arbitrarily long contexts by placing the input in a persistent REPL environment, allowing the model to write code that decomposes, queries, and aggregates over the input programmatically. We train the first open-weight natively recursive language model based on Qwen3.5-35B-A3B, a mixture-of-experts model with 35B total / 3B active parameters. Using the Tinker training API, we develop a complete RL pipeline through 11 training iterations—addressing reward collapse, mode collapse, KL regularization, importance sampling, and credit assignment—and systematically compare two training paradigms: Group Relative Policy Optimization (GRPO) with our novel strategy conditioning (SC-GRPO), and Rejection Sampling SFT (RS-SFT) on iteratively mined correct trajectories.

Our best model, trained with RS-SFT on 3,644 self-mined trajectories (no teacher model required), achieves $+21.7$pp average improvement across 14 benchmarks ($49.9\% \rightarrow 71.6\%$), winning 13 of 14 benchmarks against the base model, with peak gains of $+60$pp on Verbatim Copy, $+56$pp on Key-Value Retrieval, and $+30$pp on Hard Multi-Hop QA. This substantially outperforms our best GRPO model ($+11.3$pp), which wins only 9 of 14.

We contribute five key findings: (1) **RS-SFT outperforms GRPO by $+10.4$pp**—training only on correct trajectories avoids the cross-task interference caused by GRPO's negative gradients, which suppress useful base-model patterns; (2) **GRPO on top of RS-SFT actively hurts**—applying RL refinement to an RS-SFT checkpoint degrades 5 of 9 evaluated benchmarks, with Code Debug collapsing from 50% to 36%; (3) **Strategy-Conditioned GRPO (SC-GRPO)** eliminates mode collapse in code-generation RL (0% group collapse vs. 60% in standard GRPO); (4) **format rigidity** as the root cause of RL regressions—RL trains rigid format-specific parsing templates that fail on output variation; (5) **iterative trajectory mining** produces monotonically improving training data, with each RS-SFT round generating a better model that mines better trajectories for the next round.

We evaluate on 14 diverse benchmarks spanning $O(1)$ retrieval, $O(K)$ multi-needle search, $O(N)$ classification/counting, $O(N^2)$ cross-document comparison, multi-hop reasoning, code debugging, and notebook analysis, plus 2 external benchmarks (OOLONG trec_coarse and LongBench-v2 CodeQA). We release all weights, code, benchmarks, and training data.

1. Introduction

A key limitation of modern language models is that their effective context window, while growing, remains insufficient for many real-world tasks involving documents spanning hundreds of thousands of tokens. Even when models accept long inputs, performance degrades with context length (Hsieh et al., 2024). Recursive Language Models (RLMs) (Zhang et al., 2026) address this by never placing the full input into the model's context window. Instead, the input is stored as a variable in a persistent Python REPL, and the model writes code to decompose, search, and aggregate over it programmatically. Sub-queries are dispatched via an `llm_query()` function, enabling recursive decomposition at arbitrary depth.

Zhang et al. (Zhang et al., 2026) showed that SFT on filtered trajectories from a 480B teacher model improved Qwen3-8B by 28.3% on long-context benchmarks. However, the optimal training strategy for RLMs—whether RL, SFT, or a combination—remained unexplored. We address this gap through a systematic RL exploration spanning **11 training iterations** across two model scales (4 GRPO versions at 1.7B, 7 iterations at 35B including SC-GRPO, DPO, and RS-SFT). Through this extensive RL development—debugging

[1]Department of Computer Science, Stanford University, Stanford, CA, USA. Correspondence to: Omar Abul-Hassan <omarah@stanford.edu>.

Proceedings of the 35th International Conference on Machine Learning, Stockholm, Sweden, PMLR 80, 2018. Copyright 2018 by the author(s).

reward collapse, mode collapse, KL regularization failures, importance sampling explosions, and credit assignment—we discover that RL's relative effectiveness is the opposite of conventional wisdom.

We train Qwen3.5-35B-A3B, a mixture-of-experts (MoE) model with 35B total and 3B active parameters, using the Tinker training API for remote GPU execution. We evaluate across 14 diverse benchmarks (plus 2 external) spanning $O(1)$ retrieval, $O(K)$ multi-needle search, $O(N)$ classification and counting, $O(N^2)$ cross-document comparison, multi-hop reasoning, code debugging, notebook analysis, and more.

Our contributions are:

1. **RS-SFT outperforms GRPO for RLM training (+21.7pp vs +11.3pp).** Rejection Sampling SFT—training only on correct, high-quality trajectories—avoids the cross-task interference caused by GRPO's negative gradients. This finding contradicts the common assumption that RL should outperform behavioral cloning, and we provide detailed analysis of *why* negative gradients are particularly destructive for multi-task code generation.

2. **GRPO on top of RS-SFT actively hurts performance.** Applying GRPO refinement to an RS-SFT checkpoint degrades 5 of 9 evaluated benchmarks, with Code Debug collapsing from 50% to 36%. This demonstrates that RL's negative gradients damage the carefully learned patterns from RS-SFT, and that the SFT→RL pipeline is not universally beneficial.

3. **Strategy-Conditioned GRPO (SC-GRPO)** eliminates mode collapse in code-generation RL by conditioning each trajectory on a randomly assigned decomposition strategy (0% group collapse vs. 60% in standard GRPO).

4. **Format rigidity as the root cause of RL regressions.** Trajectory analysis reveals that RL trains rigid format-specific parsing patterns in sub-call code (e.g., `if "ORG: " in line`) that fail on output format variation. RS-SFT avoids this because it never pushes the model *away* from flexible base-model patterns.

5. **Iterative trajectory mining as a data flywheel.** Each RS-SFT round produces a better model that generates better trajectories for the next round: V16 (2,589 samples, 70.4%) → V17 (3,644 samples, 71.6%). Content-based deduplication and task-balanced batching enable this iterative self-improvement without a teacher model.

6. **16 diverse benchmarks** including 5 novel synthetic benchmarks (DataFrame QA, Event Counting, Cross-Document Comparison, Key-Value Retrieval, Notebook QA) and 2 external benchmarks from the original RLM paper (OOLONG, LongBench-v2 CodeQA). Complete open-source release of model weights, training code, evaluation harness, and all benchmark generators.

2. Background and Related Work

2.1. Recursive Language Models

RLMs (Zhang et al., 2026) treat the language model as a controller that writes and executes code in a persistent REPL environment. Given input P, the RLM loop proceeds as:

1. Initialize REPL with P stored as the variable `context`. The model receives only *metadata*: the length of P, a short prefix, and the available API (`context`, `llm_query()`, `FINAL()`, `FINAL_VAR()`).

2. The model generates Python code to process `context` (e.g., chunking, regex, slicing).

3. Sub-calls via `llm_query(text)` invoke the same model on a substring of P.

4. The model terminates with `FINAL(answer)` or `FINAL_VAR(var_name)`.

This design avoids the three anti-patterns of naive scaffolding: the full prompt never enters the context window, output is not bounded by the window, and sub-calls are programmatic rather than verbalized (Zhang et al., 2026). Algorithm 1 formalizes this loop. The key recursive primitive is `llm_query(text)`: when the model's generated code calls this function, a fresh instance of $\mathcal{M}$ is invoked on the given text, enabling recursive decomposition at arbitrary depth. Sub-calls use a fixed low temperature (0.3) for reliable factual extraction, regardless of the root generation temperature.

2.2. Reinforcement Learning from Outcome Reward

RL from outcome-level reward has shown strong results for improving reasoning in language models. DeepSeek-R1 (DeepSeek-AI, 2025) introduced Group Relative Policy Optimization (GRPO), which eliminates the critic network by computing advantages relative to a group of sampled responses.

Given prompt x and K sampled responses $\{y_1, \ldots, y_K\}$ with rewards $\{r_1, \ldots, r_K\}$, GRPO computes:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\})}{\text{std}(\{r_j\}) + \epsilon} \tag{1}$$

Algorithm 1 RLM Inference Loop

Require: Input text P, model $\mathcal{M}$, max turns T
1: `repl` $\leftarrow$ InitREPL(P) {Store P as `context`}
2: $m_0 \leftarrow$ Metadata(`repl`) {$|P|$, prefix, API docs}
3: $H \leftarrow$ [`sys_prompt`, User(m_0)]
4: **for** $t = 1\ T$ **do**
5: $c_t \leftarrow \mathcal{M}$.generate($H$) {Generate Python code}
6: $o_t, a_t \leftarrow$ Execute(`repl`, c_t) {Run in sandboxed REPL}
7: **if** $a_t \neq \perp$ **then**
a_t {`FINAL()` was called}
8: **end if**
9: $H \leftarrow H \cup$ [Asst(c_t), User(OutMeta(o_t))]
10: **end for**timeout

and updates the policy with a REINFORCE-style (Williams, 1992) gradient weighted by $\hat{A}_i$.

2.3. Rejection Sampling and Best-of-N Training

Rejection Sampling Fine-Tuning (RFT) (Yuan et al., 2024) and related approaches (Cobbe et al., 2021; Singh et al., 2024) generate multiple candidate solutions, filter for correctness, and train on only the correct ones using standard supervised learning. Unlike RL, these approaches use only *positive* signal—the model learns to reproduce correct behavior but is never explicitly pushed away from incorrect behavior. This distinction proves critical in our setting, as we show that the negative gradients in GRPO cause cross-task interference that RS-SFT avoids entirely.

2.4. Related Work

Tool-augmented LMs. ReAct (Yao et al., 2023) and Toolformer (Schick et al., 2023) augment models with external tools but keep the prompt in the context window. RLMs differ by placing the prompt *outside* the window entirely.

Self-improvement. STaR (Zelikman et al., 2022) bootstraps reasoning by generating rationales, filtering correct ones, and re-training. Our iterative trajectory mining follows a similar approach: each RS-SFT round generates trajectories from the improved model, mines the correct ones, and retrains.

Long-context evaluation. RULER (Hsieh et al., 2024) and Needle-in-a-Haystack (Kamradt, 2023) test retrieval at various context lengths. OOLONG (Bertsch et al., 2025a) provides tasks requiring processing *all* documents, not just retrieval. We design benchmarks spanning both retrieval ($O(1)$, $O(K)$) and processing ($O(N)$, $O(N^2)$) complexity.

LoRA fine-tuning. Low-Rank Adaptation (Hu et al., 2022) enables efficient fine-tuning of large models. We use LoRA for both SFT and RL stages.

Advanced RL objectives for LLMs. Dr. GRPO (Liu et al., 2025) removes standard deviation normalization and length normalization from GRPO's advantage computation, preventing length bias and entropy collapse. We incorporate this insight by using raw advantages (without standard deviation division) in our SC-GRPO implementation after observing that normalized advantages amplify noise in low-variance code-generation settings where most trajectories produce near-identical code.

Teacherless self-improvement for RLMs. The original RLM work (Zhang et al., 2026) relies on gold trajectories generated by a 480B-parameter teacher model (GPT-5 via API), achieving +28.3% average improvement on Qwen3-8B across their benchmark suite. In contrast, our approach is entirely *teacherless*: we mine correct trajectories from the base model's own evaluation runs, without requiring access to any external or proprietary model. Despite this constraint, we achieve +21.7pp average improvement on a more diverse 14-benchmark suite—demonstrating that iterative self-mining can approach teacher-distilled performance. This makes our training recipe fully self-contained and reproducible by anyone with API access to the base model.

3. Approach

Our training pipeline evolves through two model scales (Qwen3-1.7B for pipeline development, Qwen3.5-35B-A3B for final results) and two training paradigms (GRPO and RS-SFT). We describe the key methods here; the full history of 1.7B experiments (SFT, STaR, GRPO v1–v4, DPO) is preserved in Appendix F for completeness.

3.1. Model and Infrastructure

Qwen3.5-35B-A3B is a mixture-of-experts model with 35B total parameters and 3B active parameters per token (Qwen Team, 2025). The MoE architecture is cost-effective on the Tinker training API—billing is per active parameter, not total—while providing 35B total parameters for rich representations.

Tinker API. All training and generation runs remotely via the Tinker API. Training uses `forward_backward("cross_entropy")` for SFT and `forward_backward("importance_sampling")` for GRPO, followed by `optim_step()`. Generation uses `sampling_client.sample()` with configurable temperature and max tokens. This removes all GPU/VRAM constraints, enabling training of models we could never fit on our local cluster.

LoRA Configuration. Rank 32, α=32, targeting all linear layers (MLP + attention + unembed), producing ∼680M trainable parameters.

3.2. Phase 1: SC-GRPO (Strategy-Conditioned GRPO)

3.2.1. GRPO Formulation

We implement GRPO (DeepSeek-AI, 2025) with PPO-style clipping (Schulman et al., 2017). Given a prompt x and K=8 sampled trajectories $\{y_1, \ldots, y_K\}$, we compute per-trajectory rewards r_i and advantages following Dr. GRPO (Liu et al., 2025), which removes standard deviation normalization:

$$\hat{A}_i = r_i - \frac{1}{K}\sum_{j=1}^{K} r_j \tag{2}$$

We found standard normalization ($\hat{A}_i = (r_i - \mu)/(\sigma + \epsilon)$) amplified advantages by up to $143\times$ in low-variance groups (e.g., rewards $[0.72, 0.73, 0.71]$ yield σ=0.007, inflating a 0.01 raw advantage to 1.43). The policy gradient loss with PPO clipping is:

$$\mathcal{L} = -\frac{1}{K}\sum_{i=1}^{K}\sum_{t=1}^{|y_i|} w_t \cdot \min\Big(\rho_t \hat{A}_i,\ \text{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon)\hat{A}_i\Big) \tag{3}$$

where $\rho_t = \pi_\theta(y_{i,t}|y_{i,<t}, x)/\pi_{\theta_{\text{old}}}(y_{i,t}|y_{i,<t}, x)$ is the importance sampling ratio, ϵ=0.2 is the clipping threshold, and w_t is the per-turn credit assignment weight (defined below). KL regularization is applied via reward shaping: $r'_i = r_i - \beta_{\text{KL}} \cdot \widehat{\text{KL}}_i$, where $\widehat{\text{KL}}_i$ is the average per-token KL divergence between the current policy and a frozen reference copy.

PPO clipping is essential. Early experiments used unclipped importance sampling (`forward_backward("importance_sampling")` in the Tinker API). This produced gradient explosions: loss values of $O(10^7)$ vs $O(10^2)$ with clipping. The root cause is that gradient accumulation over 48–55 datums per step allows the policy to drift *within* a training step, causing IS ratios ρ_t to diverge before the update. Switching to PPO-clipped loss (`forward_backward("ppo")`) resolved this completely.

KL regularization: weight-space vs. token-level. We initially attempted weight-space L2 regularization ($\beta\|\theta - \theta_{\text{ref}}\|^2/d$) as KL proxy. With LoRA's 17.4M parameters, per-parameter L2 values were $O(10^{-10})$, producing effectively zero regularization even at β=0.02. This caused catastrophic policy drift: Multi-NIAH collapsed from 97.8% to 41.4% in 10 steps. Switching to token-level KL via a frozen reference forward pass (computing $\sum_t \log \pi_{\text{ref}}(y_t|y_{<t}) - \log \pi_\theta(y_t|y_{<t})$ and subtracting from reward) resolved the drift while preserving learning signal.

3.2.2. Mode Collapse in Code-Generation GRPO

Standard GRPO applied to code generation suffers from severe mode collapse: within 5 steps, 60% of trajectory groups produce identical code across all K=8 samples, yielding zero advantages and no gradient signal. We quantify this in our V6 experiments: of 5 trajectory groups per step, 3 had standard deviation 0.000 across all K=8 rewards (identical code, identical outcomes). This is fundamentally different from natural language generation—code has a much narrower mode space, and once the model finds one working template (e.g., scan-all-chunks-aggregate), it converges to that template character-for-character.

The collapse timeline in our V1 experiments at 1.7B scale (Section F) is illustrative: steps 1–10 produced 15–24 parameter updates per step (healthy diversity), steps 11–17 produced 8–16 updates (converging), and steps 18–30 produced **0 updates** (complete stagnation). All K trajectories became identical, advantages were uniformly zero, and training halted despite continued GPU computation.

3.2.3. Strategy Conditioning

SC-GRPO. We address mode collapse by prepending a randomly assigned *strategy prompt* to each trajectory's system prompt. We define 10 strategies spanning different decomposition approaches:

1. **Standard**: default chunking and aggregation
2. **Extract-Compute**: extract raw data, compute answer in Python
3. **Binary Search**: narrow down location via successive halving
4. **Map-Reduce**: map sub-queries over chunks, reduce results
5. **Two-Pass**: scan pass for candidates, verify pass to confirm
6. **Small Chunks**: 5K chunks with high overlap for precision
7. **Table Preserve**: preserve CSV/tabular structure in chunks
8. **Notebook Sequential**: process cells in order, track state
9. **Lookup Thorough**: exhaustive small-chunk search for exact values
10. **Precision Extract**: extract exact text, not summaries

Each trajectory in a group of K=8 is randomly assigned one strategy. This forces structural diversity: even if the model prefers one approach, the strategy prompt pushes toward alternatives. The result is 0% group collapse, ensuring every training step has meaningful gradient signal.

Task-specific strategy distributions. Different task types benefit from different strategy mixtures. For event counting, we weight extract-compute (40%), map-reduce (30%), and small-chunks (20%). For key-value retrieval, we weight lookup-thorough (40%) and binary-search (25%). These distributions are tuned per-task type based on which strategies produce the highest-reward trajectories.

SC-GRPO hyperparameters. K=8 trajectories per prompt, batch size 4 tasks, gradient accumulation 4, LoRA rank 32, learning rate 1×10^{-6} (cosine decay to 10% of peak), KL coefficient 0.005 via reward shaping. Temperature schedule $[0.6, 0.7, \ldots, 1.3]$ across the 8 trajectories. Fixed sub-call temperature of 0.3.

Reward function. Reward design is critical for RL in code generation, where sparse binary rewards cause reward collapse (Section F). We use a composite reward with task-specific weighting:

$$r = \alpha\cdot r_{\text{score}} + \beta\cdot r_{\text{format}} + \gamma\cdot r_{\text{persist}} + \delta\cdot r_{\text{subcall}} + \zeta\cdot r_{\text{decomp}} \quad (4)$$

where $r_{\text{score}} \in [0, 1]$ is the task-specific correctness metric (F1 for multi-NIAH, accuracy for doc-classify, exact match for NIAH), $r_{\text{format}} = +0.05$ if the trajectory terminates with `FINAL()`, $r_{\text{persist}} \in [0.05, 0.15]$ rewards multi-turn correct solutions (encouraging iterative refinement over single-pass attempts), $r_{\text{subcall}} = +0.10$ for trajectories using 2+ `llm_query` calls on long contexts (preventing shortcut solutions that avoid recursion), and r_{decomp} provides partial credit for bridge entities in multi-hop tasks (rewarding intermediate progress even if the final answer is wrong). The weights $(\alpha, \beta, \gamma, \delta, \zeta)$ vary by task type: NIAH uses $(0.75, 0.10, 0.10, 0.05, 0)$; hard multi-hop uses $(0.50, 0.10, 0.10, 0.10, 0.20)$.

Per-turn credit assignment. RLM trajectories are multi-turn: a single trajectory may contain 3–8 code blocks (turns), each with different semantic roles. Applying uniform advantage $\hat{A}_i$ to all turns dilutes the learning signal, since a correct final turn should receive more credit than a boilerplate setup turn. We assign per-turn weights w_t in Equation 3:

For *successful* trajectories ($r_i \geq 0.5$): FINAL turns receive $1.5\times$, turns containing `llm_query` sub-calls receive $1.0\times$, setup/print turns receive $0.7\times$, and error turns (in otherwise-successful trajectories) receive $0.3\times$. For *failed* trajectories: timeout turns receive $1.5\times$ blame, error turns $1.2\times$, FINAL turns $1.3\times$, others $1.0\times$. All weights are normalized to mean 1.0 per trajectory to preserve total gradient magnitude:

$$w_t^{\text{norm}} = w_t^{\text{raw}} \cdot \frac{|y_i|}{\sum_{t'=1}^{|y_i|} w_{t'}^{\text{raw}}} \quad (5)$$

This credit assignment is a form of structured reward shaping (Schulman et al., 2017): it preserves the optimal policy (since the advantage magnitudes are unchanged in aggregate) while focusing gradient signal on the turns that most influence the outcome.

Regression-targeted task distribution. After observing a zero-sum tradeoff in early GRPO runs (gains on some benchmarks cancelled by regressions on others), we allocate 70% of training weight to previously-regressing tasks: cross-document comparison (20%), key-value retrieval (14%), DataFrame QA (14%), event counting (12%), with the remaining 30% split across NIAH (10%), doc-classify (10%), hard multi-hop (10%), and code debug (10%).

V10-s40: Our best GRPO model, trained for 40 steps from the V4-s5 checkpoint (which was 5 standard GRPO steps from the base model). Total training time: $\sim$20 hours on Tinker API.

3.3. Phase 2: Hybrid RLM Architecture

We discover that RL training degrades sub-call QA quality because LoRA adapts both root code generation and sub-call response distributions, but training rewards only the final answer. This creates an unintended coupling between *planning* (root code generation) and *perception* (sub-call fact extraction): improving the model's code generation simultaneously biases its text comprehension, because both share the same LoRA adapter.

The **hybrid architecture** addresses this by maintaining two model instances: a trained root model (with LoRA adapter) for code generation, and the base model (no adapter) for answering sub-call queries. This separation preserves the base model's general-purpose QA ability while allowing RL to optimize code generation.

Trajectory analysis reveals the mechanism: RL-trained sub-calls attempt to answer questions using format-specific patterns learned during training (e.g., looking for "ORG: Name" format), while base-model sub-calls use flexible extraction that adapts to varying output formats. In our GRPO experiments, the hybrid architecture achieves +25pp over standard (non-hybrid) trained models on multi-hop reasoning, because base-model sub-calls answer atomic factual queries ("Who manages HR?") more reliably than RL-biased sub-calls, which learn shortcuts like returning the first entity found.

Implications for RS-SFT. RS-SFT partially mitigates planning-perception entanglement by training only on trajectories where both root code and sub-call responses produce correct results. This self-consistency filtering naturally excludes trajectories with poor sub-call extraction, reducing (but not eliminating) the coupling. V17 RS-SFT achieves strong results *without* hybrid inference, suggesting that positive-only training is less destructive to sub-call quality than GRPO's negative gradients. Nevertheless, hybrid inference remains a promising direction for further gains,

particularly on $O(N^2)$ cross-document tasks where sub-call quality is the primary bottleneck.

3.4. Phase 3: Rejection Sampling SFT (RS-SFT)

The central finding of this work: **RS-SFT substantially outperforms GRPO for RLM training.**

3.4.1. Motivation: Why Negative Gradients Hurt

GRPO trains on *all* K trajectories per prompt, weighting each by its advantage $\hat{A}_i$. Trajectories with positive advantage receive positive gradients (reinforce these patterns); trajectories with negative advantage receive negative gradients (suppress these patterns).

The problem is that negative gradients cause **cross-task interference**: a code pattern that appears in a "bad" trajectory for one task type may be the *same pattern* the base model uses successfully for a different task type. Suppressing it hurts globally. For example, GRPO may learn to suppress loose text parsing (e.g., `for line in answer.split("\n")`) because it appeared in a failed notebook QA trajectory, even though this same pattern is essential for successful event counting.

RS-SFT eliminates this problem entirely by training only on correct trajectories with standard cross-entropy loss. The model can only learn to reproduce exemplary behavior—it is never pushed *away* from anything. This makes it theoretically impossible for RS-SFT to cause the format rigidity regression observed with GRPO (Section 6.2).

3.4.2. Data Collection: Iterative Trajectory Mining

Rather than collecting trajectories from a 480B-parameter teacher model (as in Zhang et al. (Zhang et al., 2026), who used GPT-5 to generate gold demonstrations), we mine correct trajectories from the base model's own evaluation runs—a fully *teacherless* approach that requires no access to any external model:

1. **Round 1.** Run the base model on all 14 benchmarks with K=8 attempts per task (seed-based generation for reproducibility). Filter trajectories with score ≥ 0.9, proper `FINAL` termination, at least one `llm_query` call, clean code (no errors), and reasonable chunk sizes (5K–25K chars). Content-based deduplication removes duplicates across tasks with the same type. Task-balanced batching ensures equal representation across all 14 benchmarks.
2. **Round 2.** Train RS-SFT on the Round 1 data (producing V16). Run V16 on all benchmarks. Mine *additional* correct trajectories—including trajectories for tasks that the base model could not solve but V16 can. Merge with Round 1 data, deduplicate, rebalance.
3. **Round 3.** Train on the merged data (producing V17). Evaluate.

This iterative mining is analogous to STaR (Zelikman et al., 2022) but operates at the trajectory level rather than the rationale level. Each round produces a better model that can solve harder tasks, generating higher-quality and more diverse training data for the next round.

3.4.3. Quality Filtering

All mined trajectories pass through 8 quality filters:

1. **Correctness**: Score ≥ 0.9 (handles accuracy, recall, F1 depending on benchmark)
2. **Proper termination**: `FINAL()` or `FINAL_VAR()` called
3. **Uses sub-calls**: At least 1 `llm_query` call (prevents memorization/direct-answer)
4. **Clean code**: No syntax errors or exceptions
5. **Turn count**: 1–8 turns (prevents runaway trajectories)
6. **Multi-turn for complex tasks**: Hard multi-hop, cross-doc tasks require 2+ turns
7. **Reasonable chunk size**: Chunks between 5K–25K chars
8. **No f-string bugs**: No `llm_query("{context}")` without f-string

3.4.4. Training Configuration

- **Loss**: Standard cross-entropy via Tinker's `forward_backward("cross_entropy")`
- **LoRA rank**: 32 (same as GRPO)
- **Learning rate**: 2×10^{-5} (10× higher than GRPO, following Tinker API recommendations for SFT)
- **Schedule**: Linear warmup for 10% of steps, cosine decay to 10% of peak LR
- **Epochs**: 3 over the filtered dataset
- **Batch size**: 4 with gradient accumulation of 4 = effective batch 16
- **Per-token weighting**: From Tinker's renderer, masking system/user tokens and weighting only assistant code tokens

V16 RS-SFT: 2,589 mined trajectories, 3 epochs, ∼2.5 hours on Tinker. Average: 70.4%.

V17 RS-SFT: 3,644 mined trajectories (includes Round 2 mining from V16), 3 epochs, ∼2.8 hours on Tinker. Average: 71.6%. **This is our best model.**

3.5. Phase 4: SFT Scaling Experiments (V18–V21)

After V17, we conducted four experiments attempting to push beyond 71.6%:

V18 (4,660 samples): We discovered a bug in trajectory mining—`mine_eval_trajectories.py` only checked the `score` field, but doc-classify uses `accuracy` and multi-NIAH uses `recall`. Fixing this added 858 doc-classify and 798 multi-NIAH trajectories. Despite more data, V18 scored 69.7% (-1.9pp). Adding doc-classify data improved doc-classify (+7.7pp over V17) but degraded code debug (-20pp) and hard multi-hop (-20pp).

V19 (V17 + 1 epoch of V18 data, lr 5×10^{-6}): Fine-tuning V17 with the new data at 10× lower learning rate. Score: 69.5% (-2.1pp). Even gentle fine-tuning damages V17's reasoning.

V20 (4,277 reasoning-boosted samples): Rebalanced data toward hard reasoning tasks. Score: 69.4% (-2.2pp). Notebook QA improved to 90% (best ever) but multi-hop QA collapsed to 45% (below base).

V21 (V17 + 1,732 targeted samples, lr 3×10^{-6}): Minimal targeted fine-tuning on V17's weakest benchmarks. Score: 64.8% (-6.8pp). Cross-doc collapsed to 18.5%.

Key insight: SFT creates unavoidable benchmark tradeoffs. Every model that gains on some benchmarks loses on others. V17's specific balance of 3,644 samples across 3 epochs represents a local optimum that is surprisingly difficult to improve upon. The oracle maximum (best per-benchmark across V17–V21) is 77.7%, indicating ∼6pp of headroom that could be unlocked by per-benchmark model routing.

4. Experimental Setup

4.1. Benchmarks

We evaluate on 14 internal benchmarks spanning four algorithmic complexity classes, plus 2 external benchmarks. All benchmarks use deterministic seed-based generation for reproducible head-to-head comparison. Training and evaluation seeds are separated by a +10,000 offset.

4.1.1. $O(1)$ Tasks (Find a Single Piece of Information)

NIAH (20 tasks). Needle-in-a-Haystack: a single piece of information (e.g., "The secret code is ALPHA-7") is hidden at a random position in a 5K–100K character document of filler text. The model must locate and extract this information. Scoring: binary (1.0 if expected value found, 0.0 otherwise).

Hard NIAH (15 tasks). Adversarial variant with three difficulty modes: (a) *distractor tasks* (40%)—similar-looking values placed near the true needle (e.g., "previous access code was BETA-22K" near "current active code is ALPHA-7X9"); (b) *extreme length* (30%)—documents of 200K–1M characters; (c) *boundary tasks* (30%)—needles placed at extreme positions (0.01, 0.02, 0.97, 0.99).

Key-Value Retrieval (12 tasks). A registry document (like a database dump) with hundreds to thousands of entries, each with `REGISTRY_ID`, `VALUE`, `STATUS`, `CATEGORY`. Three task types: single lookup (find one entry by ID), multi-lookup (find 3–5 entries), and aggregate (sum values matching a category filter). Context: 50K–200K characters.

Verbatim Copy (10 tasks). The model must locate and *exactly reproduce* a specific paragraph (∼150–250 characters) from a 10K–100K character document. Scoring: character-level similarity via `SequenceMatcher` (≥95% = 1.0, ≥80% = 0.75, ≥60% = 0.5).

4.1.2. $O(K)$ Tasks (Find Multiple Scattered Items)

Multi-NIAH (20 tasks). K=3–10 needles embedded at different positions in 10K–100K character documents. Each needle formatted as `SECRET_CODE_[NAME]:[VALUE]`. Scoring: F1 over found needles.

Multi-Hop QA (20 tasks). Questions requiring 2–3 reasoning hops across different parts of a 10K–100K character document (e.g., "Find the person in role X, then find their project, then find its budget"). Facts are placed at separated positions to ensure they fall in different chunks. Scoring: exact/contains match (1.0), word overlap ≥70% (0.5).

Hard Multi-Hop QA (10 tasks). Harder variant with explicit distractor chains in 100K–200K character documents. A target chain (e.g., "VP of Engineering is Rachel Thomas" → "Rachel Thomas leads Project Phoenix" → "Project Phoenix delivery date is March 15, 2026") is accompanied by a distractor chain with similar structure but different entities. Facts are placed far apart so the model *cannot* concatenate the full answer into a single chunk.

Code Debug (15 tasks). A Python codebase (200–1000 lines) with 1–3 planted bugs (off-by-one errors, logic errors, index swaps, wrong operators). The model must locate and describe each bug. Scoring: fraction of bugs correctly identified (must mention function name + evidence of understanding).

Notebook QA (15 tasks). Simulated Jupyter notebooks

(15K–60K characters) with interleaved code cells, markdown, and outputs. Three task types: output lookup (find a specific cell's output), variable trace (follow a variable across cells), and cross-cell (compare results from different cells).

4.1.3. $O(N)$ Tasks (Process Every Item)

Doc-Classify (20 tasks). Classify N=5–20 synthetic news articles into 6 categories (Science, Sports, Business, World, Health, Technology). Each article is ∼300–800 characters. The model must process every document and return per-document classifications.

DataFrame QA (20 tasks). A synthetic stock price dataset (5–50 tickers, 30–250 trading days) presented as CSV text (8K–700K characters). Task types: lookup (find a specific price), aggregation (compute average/max), ranking (which stock has highest volatility?), sector analysis, and multi-step queries.

Event Counting (20 tasks). Event logs (50K–200K characters) with 30–100 events scattered among 16–20 entities and 15 event types. Five task types: count by value, count by entity, first/last event, per-entity breakdown, and ratio computation. The correct strategy is to *extract* matching events via sub-calls, then count/aggregate in Python—delegating counting to sub-calls produces hallucinated numbers.

4.1.4. $O(N^2)$ Tasks (Cross-Document Comparison)

Cross-Doc Compare (12 tasks). Two documents (30K–100K characters each, embedded in a single context) require cross-document analysis. Four task types: entity overlap (find employees in both org charts), budget difference (largest absolute budget diff across shared projects), timeline conflict (events with conflicting dates), and metric comparison (which division has better average metrics?).

4.1.5. External Benchmarks

OOLONG trec_coarse (10 tasks, ∼128K tokens). From Bertsch et al. (Bertsch et al., 2025b). Document classification tasks requiring label frequency counting, comparison, and aggregation. Scoring follows the original paper: $0.75^{|y-\hat{y}|}$ for numeric answers, exact match otherwise.

LongBench-v2 CodeQA (15 tasks, ≤500K characters). From Bai et al. (Bai et al., 2024). Multiple-choice code repository understanding questions. The original RLM paper reported RLM(GPT-5) at 62% on this benchmark.

4.2. Models Compared

1. **Base**: Qwen3.5-35B-A3B with RLM scaffold, no fine-tuning.
2. **V10-s40 (SC-GRPO)**: 40 SC-GRPO steps from V4-s5 checkpoint. LoRA rank 32, lr 1×10^{-6}, regression-targeted task distribution. Best GRPO model.
3. **V16 (RS-SFT)**: RS-SFT on 2,589 mined trajectories. 3 epochs, lr 2×10^{-5}.
4. **V16+GRPO**: 10 GRPO steps on top of V16 RS-SFT checkpoint. Tests whether RL refines RS-SFT.
5. **V17 (RS-SFT)**: RS-SFT on 3,644 mined trajectories (includes Round 2 mining from V16). 3 epochs, lr 2×10^{-5}. **Best overall model.**
6. **V18–V21 (ablations)**: SFT scaling experiments (Section 3.5).

All results use seed 10000 for fair head-to-head comparison.

4.3. Computational Cost

Phase	API Hours
Base model evaluation (14 benchmarks)	∼3.0
V4-s5 GRPO (5 steps from base)	∼2.5
V10-s40 SC-GRPO (40 steps from V4-s5)	∼20.0
V10-s40 evaluation	∼3.0
Trajectory mining (3 rounds, 14 benchmarks)	∼15.0
V16 RS-SFT training (2,589 samples, 3 ep.)	∼2.5
V17 RS-SFT training (3,644 samples, 3 ep.)	∼2.8
V16+GRPO training (10 steps)	∼5.0
V18–V21 experiments	∼10.0
All evaluations (×8+ models)	∼20.0
Total	**∼84**

All Qwen3.5-35B-A3B experiments use the Tinker API for remote GPU execution. The 1.7B pipeline development phase (Appendix F) used ∼11.1 GPU-hours on a single NVIDIA H200.

5. Results

5.1. Main Results

Table 1 shows the primary comparison across all 14 benchmarks. We compare the base model, V10-s40 (best GRPO), and V17 (best RS-SFT). All results use seed 10000 in clean head-to-head evaluation (identical seeds, no strategy prompts).

V17 RS-SFT achieves +21.7pp average improvement, winning 13 of 14 benchmarks against the base model. The only regression is Multi-NIAH (−4.3pp, from 99.4% to 95.1%—still very high). Against V10-s40 (best GRPO), V17 wins 12 of 14.

The largest gains come from tasks requiring precise extraction and multi-step reasoning. Verbatim Copy

Table 1. Head-to-head evaluation across 14 benchmarks (seed 10000). Base = Qwen3.5-35B-A3B with RLM scaffold, no fine-tuning. V10-s40 = best SC-GRPO model (40 steps, regression-targeted). V17 = best RS-SFT model (3,644 trajectories, 3 epochs). Best per-benchmark in **bold**. ↑ = improvement over base, ↓ = regression.

	Benchmark (n)	Base	V10-s40	Δ Base	**V17**	Δ Base	Δ V10
$O(1)$	NIAH (20)	65.0	75.0	↑10.0	**75.0**	↑10.0	0.0
	Hard NIAH (15)	83.3	93.3	↑10.0	**96.7**	↑13.4	↑3.4
	KV Retrieval (12)	29.2	75.0	↑45.8	**85.4**	↑56.2	↑10.4
	Verbatim Copy (10)	20.0	60.0	↑40.0	**80.0**	↑60.0	↑20.0
$O(K)$	Multi-NIAH (20)	**99.4**	90.0	↓9.4	95.1	↓4.3	↑5.1
	Multi-Hop QA (20)	55.0	60.0	↑5.0	**80.0**	↑25.0	↑20.0
	Hard Multi-Hop (10)	30.0	30.0	0.0	**60.0**	↑30.0	↑30.0
	Code Debug (15)	50.0	43.3	↓6.7	**60.0**	↑10.0	↑16.7
	Notebook QA (15)	46.7	63.3	↑16.6	**70.0**	↑23.3	↑6.7
$O(N)$	Doc-Classify (20)	56.3	76.6	↑20.3	**80.2**	↑23.9	↑3.6
	DataFrame QA (20)	75.0	85.0	↑10.0	**93.0**	↑18.0	↑8.0
	Event Counting (20)	46.4	41.7	↓4.7	**73.0**	↑26.6	↑31.3
$O(N^2)$	Cross-Doc (12)	42.2	42.9	↑0.7	**43.4**	↑1.2	↑0.5
External	OOLONG (10)	0.0	**20.0**	↑20.0	10.0	↑10.0	↓10.0
Average (14)		49.9	61.1	↑11.3	**71.6**	**↑21.7**	**↑10.4**
Win/Loss/Tie vs Base		—	9/3/2	—	**13/1/0**	—	—
Win/Loss/Tie vs V10-s40		—	—	—	**12/1/1**	—	—

Table 2. Training progression across methods. All averages over 14 benchmarks, seed 10000.

Model	Method	Data	Avg (%)
Base	—	—	49.9
V10-s40	SC-GRPO	592 prompts	61.1
V16	RS-SFT	2,589 mined	70.4
V17	**RS-SFT**	**3,644 mined**	**71.6**
V18	RS-SFT	4,660 (+bug fix)	69.7
V19	V17+finetune	V18 data	69.5
V20	RS-SFT	4,277 rebalanced	69.4
V21	V17+targeted	1,732 targeted	64.8

(+60pp), KV Retrieval (+56pp), Hard Multi-Hop (+30pp), Event Counting (+26.6pp), Multi-Hop QA (+25pp), and Doc-Classify (+23.9pp) all improve dramatically.

V17 fixes V10-s40's regressions. V10-s40 regressed on 3 benchmarks (Multi-NIAH −9.4pp, Code Debug −6.7pp, Event Counting −4.7pp). V17 fixes all three: Code Debug 43.3% → 60.0%, Event Counting 41.7% → 73.0%, Multi-NIAH 90.0% → 95.1%. This is the core advantage of RS-SFT over GRPO—it avoids the regressions that negative gradients cause.

5.2. Training Progression

Table 2 shows the progression from base model through GRPO to RS-SFT.

The progression is clear: RS-SFT produces a large jump over GRPO (+9.3pp from V10-s40 to V16), and iterative mining yields a further +1.2pp (V16 to V17). However, further scaling (V18–V21) does not improve beyond V17, suggesting diminishing returns from additional SFT data.

5.3. RS-SFT vs GRPO: The Core Finding

The comparison between V10-s40 (best GRPO) and V17 (best RS-SFT) reveals a systematic pattern (Table 1):

V17 wins on 12 of 14 benchmarks vs V10-s40. The most dramatic improvements are on tasks where GRPO *regressed*: Event Counting (+31.3pp), Hard Multi-Hop (+30pp), Verbatim Copy (+20pp), Multi-Hop QA (+20pp), Code Debug (+16.7pp). These are precisely the tasks where GRPO's negative gradients suppressed useful base-model patterns.

V10-s40 wins only on OOLONG (+10pp over V17). OOLONG is an external benchmark requiring real-world document aggregation at 128K tokens—the one task type least represented in the RS-SFT training data.

NIAH ties at 75.0%. Both methods improve equally on single-needle retrieval.

5.4. GRPO on Top of RS-SFT: A Negative Result

A natural question is whether RL can *refine* an RS-SFT checkpoint. We test this by applying 10 GRPO steps to the V16 RS-SFT checkpoint. Table 3 shows the results on the 9 benchmarks evaluated.

GRPO on top of RS-SFT degrades average performance

Table 3. GRPO applied on top of V16 RS-SFT. On the 9 shared benchmarks, GRPO degrades average performance. Benchmarks where GRPO hurts are shown in red.

Benchmark	V16 RS-SFT	V16+GRPO
NIAH	65.0	65.0
Multi-NIAH	95.0	96.1
Hard NIAH	**83.3**	83.3
Doc-Classify	76.7	**83.6**
DataFrame QA	**99.0**	94.0
Code Debug	**50.0**	35.6
Multi-Hop QA	75.0	75.0
Notebook QA	**83.3**	73.3
Verbatim Copy	**100.0**	80.0
Average (9)	**80.8**	76.2

by −4.6pp. Code Debug collapses from 50.0% to 35.6% (−14.4pp), Verbatim Copy from 100% to 80% (−20pp), DataFrame QA from 99% to 94% (−5pp), and Notebook QA from 83.3% to 73.3% (−10pp). The only benefit is a modest doc-classify improvement (+6.9pp). This demonstrates that **the SFT→RL pipeline is not universally beneficial**—for multi-task code generation, RL's negative gradients damage the carefully learned patterns from RS-SFT.

5.5. SFT Scaling: Diminishing Returns (V18–V21)

Table 4 shows per-benchmark results for the SFT scaling experiments.

The oracle column shows the **theoretical ceiling at 77.7%**—the average if we could select the best model per benchmark. This 6.1pp gap between V17 (71.6%) and the oracle suggests that per-benchmark model routing or mixture-of-experts at the trajectory level could yield further improvements.

Reasoning tasks are the most fragile. Code Debug and Hard Multi-Hop are the best indicators of model quality: V17 achieves 60% on both, while every other variant drops to 40–46%. These tasks require genuine multi-step decomposition, and even small changes to the training data can disrupt this capability.

5.6. External Benchmarks

V10-s40 doubles the base model's accuracy on LongBench-v2 CodeQA (+13.4pp), demonstrating transfer to real-world code comprehension. On OOLONG, the base model performs better (−7.7pp), consistent with the specialization-generalization tradeoff. The 128K-token OOLONG contexts exceed the model's 65K context window, and V10's more aggressive strategies sometimes cause sub-call context overflow.

5.7. Strategy Prompts at Evaluation Time

While SC-GRPO uses strategy prompts during *training*, we also test strategy prompts at *inference*. Per-benchmark strategy selection yields strong task-specific gains:

- Hard Multi-Hop: +40pp with `multi_hop_decompose`
- Code Debug: +19pp with `code_debug_focused`
- DataFrame QA: +17pp with `table_preserve`
- KV Retrieval: +17pp with `lookup_thorough`

However, strategies catastrophically hurt free-form tasks (Multi-Hop QA: −25pp, Verbatim Copy: −60pp). **Per-benchmark strategy selection is critical**—a single "best" strategy does not exist. The same model weights can produce 47% vs 80% depending on the strategy prompt.

Strategy amplification in GRPO. Decomposing the total improvement in our GRPO experiments (V9-s10) reveals that training alone contributes only +0.9pp average, while strategy prompts at evaluation time contribute +5.5pp—a 6× difference. This motivated our shift toward RS-SFT: since GRPO's direct behavioral change is near zero-sum across benchmarks, the question becomes whether a different training paradigm can achieve larger direct improvements. RS-SFT's +21.7pp demonstrates that positive-only training succeeds where GRPO's negative gradients create a zero-sum tradeoff, achieving large gains *without* requiring strategy prompt engineering at evaluation time.

6. Discussion

6.1. Why RS-SFT Outperforms GRPO: An RL Analysis

The +10.4pp advantage of RS-SFT over GRPO is our most important finding. Understanding *why* policy gradient methods underperform behavioral cloning in this setting provides insights for the broader RL-for-LLM community. We identify three mechanisms, each rooted in fundamental properties of the RL objective:

1. Cross-task interference via negative gradients. GRPO updates the policy on all K trajectories, including those with negative advantages. A code pattern appearing in a "bad" trajectory for task A (receiving negative gradient) may be the same pattern used successfully for task B. Suppressing it hurts B. RS-SFT never pushes the model away from anything, avoiding this interference entirely.

2. Format rigidity. Trajectory analysis reveals that GRPO trains rigid format-specific parsing in sub-call code (Section 6.2). For example, after GRPO training, the model generates `if "ORG: " in line: parts =`

Table 4. SFT scaling experiments (V18–V21) compared to V17. All trained from base unless noted. Per-benchmark best across V17–V21 in **bold**. Oracle = best per-benchmark.

Benchmark	V17	V18	V19	V20	V21	Oracle
NIAH	75.0	**80.0**	70.0	70.0	70.0	80.0
Multi-NIAH	95.1	93.4	**96.8**	95.0	88.1	96.8
Hard NIAH	**96.7**	90.0	93.3	83.3	80.0	96.7
Doc-Classify	80.2	**87.9**	78.0	84.7	84.8	87.9
DataFrame QA	93.0	94.0	97.0	94.0	**99.0**	99.0
Code Debug	**60.0**	40.0	46.7	43.3	43.3	60.0
Multi-Hop QA	**80.0**	70.0	70.0	45.0	75.0	80.0
Hard Multi-Hop	**60.0**	40.0	40.0	50.0	40.0	60.0
Notebook QA	70.0	76.7	83.3	**90.0**	80.0	90.0
Event Counting	**73.0**	61.6	57.2	67.0	60.3	73.0
Cross-Doc	43.4	**47.0**	34.7	45.4	18.5	47.0
KV Retrieval	85.4	75.0	86.7	**88.9**	63.9	88.9
Verbatim Copy	80.0	90.0	**100.0**	**100.0**	90.0	100.0
OOLONG	10.0	**30.0**	20.0	15.0	15.0	30.0
Average	**71.6**	69.7	69.5	69.4	64.8	**77.7**

Table 5. External benchmarks from the original RLM paper.

Benchmark	Base	V10-s40	Δ
OOLONG trec_coarse (50)	**32.0**	24.3	−7.7
LongBench-v2 CodeQA (15)	13.3	**26.7**	+13.4

`line.split("ORG: ")` instead of the base model's robust `for line in answer.split("\n")`. When sub-call output doesn't match the expected format, extraction fails silently.

3. Zero-sum tradeoff. GRPO training consistently shows a near-zero-sum pattern: 5 benchmarks improve, 5 regress. The regression-targeted task distribution (V10-s40) partially breaks this deadlock by allocating 70% of training weight to regressing tasks, but still produces 3 regressions. RS-SFT achieves 13 wins with only 1 minor regression (Multi-NIAH, from 99.4% to 95.1%).

6.2. Format Rigidity: The Root Cause of RL Regressions

We conduct trajectory analysis comparing base model and GRPO-trained model outputs on cross-document comparison tasks (where GRPO regresses).

Base model (43% accuracy) uses loose extraction:

```
for line in answer.split("\n"):
    line = line.strip()
    if line:
        results.append(line)
```

GRPO-trained model (28.6% accuracy) uses strict format parsing:

```
if "ORG: " in line:
    parts = line.split("ORG: ")
    org_name = parts[1].strip()
```

The GRPO model's code *works on average* during training because sub-calls often produce "ORG: Name" format. But at evaluation time, sub-call outputs vary ("Organization: Name", "The organization is Name", etc.), causing silent extraction failures.

Root causes: (1) RL rewards code that works "on average" even if brittle; (2) template convergence—seeing the same task type multiple times drives the model toward a single winning template; (3) no negative signal for format failures—the model doesn't learn *why* format-specific parsing fails; (4) sub-call non-determinism biases training toward templates that work "most of the time."

Why RS-SFT avoids this: RS-SFT quality filtering explicitly selects trajectories with format-robust parsing. The 8-criterion filter (Section 3.4) rejects trajectories with strict format-specific patterns, ensuring the model only trains on code that uses loose, flexible extraction.

6.3. Sub-Call Scaling Failure

Beyond format rigidity, GRPO training introduces a structural regression on tasks with large structured data. On DataFrame QA, the trained model learns to delegate CSV parsing to `llm_query` sub-calls—a pattern that works for small datasets but catastrophically fails for large ones because the sub-call context window cannot accommodate the full table:

Data size	Base	GRPO	Failure mode
5t/30d (7K chars)	40%	**76%**	—
15t/60d (54K chars)	40%	20%	Sub-call timeout
30t/120d (217K chars)	**80%**	∼0%	Context overflow
50t/250d (753K chars)	**56%**	∼0%	Context overflow

The base model avoids this by parsing CSV directly in Python (`for line in context.split("\n")`), which scales to arbitrary sizes. RL rewards the `llm_query` delegation pattern because it works "on average" during training (where most training examples use moderate-sized contexts), even though it has structural scaling failures outside the training distribution. RS-SFT avoids this because successful trajectories on large DataFrames naturally use Python-based parsing—the quality filter selects for patterns that actually scale.

6.4. Training Benefits by Algorithmic Complexity

Stratifying V17's improvement by benchmark complexity class (Table 11, Appendix L) reveals a systematic pattern: $O(1)$ tasks (precise retrieval) benefit most (+34.6pp average), $O(N)$ tasks (process every item) gain +22.9pp, $O(K)$ tasks (multi-item search) gain +14.8pp, while $O(N^2)$ tasks (cross-document comparison) gain only +1.2pp. This gradient suggests that RS-SFT primarily teaches systematic search patterns and extraction validation—capabilities whose benefit scales with the number of items to find—but does not yet address the combinatorial challenge of cross-document reasoning, where the model must jointly reason about information from two separate sources. The $O(N^2)$ gap represents the most promising direction for future improvement.

6.5. The SFT Scaling Frontier

The V18–V21 experiments reveal a fundamental limitation of single-model SFT:

Every model that gains on some benchmarks loses on others. V18's doc-classify gains (+7.7pp over V17) come at the cost of code debug (−20pp) and hard multi-hop (−20pp). V20's notebook QA gains (+20pp) come at the cost of multi-hop QA (−35pp). Even V21's minimal targeted fine-tuning (1,732 samples at lr 3×10^{-6}) causes cross-doc to collapse from 43.4% to 18.5%.

The oracle analysis quantifies the frontier. At 77.7% average (best per-benchmark across V17–V21), there is ∼6pp of headroom. This could be unlocked by per-benchmark model routing, mixture-of-experts at the trajectory level, or task-conditional adapters.

Reasoning tasks as canaries. Code Debug and Hard Multi-Hop are the most sensitive indicators of training quality. Their fragility to data changes suggests they test capabilities (multi-step decomposition, precise reasoning) that are easily disrupted by SFT on diverse data.

Table 6. Comparison with the original RLM training approach.

	Zhang et al.	Ours
Base model	Qwen3-8B	Qwen3.5-35B-A3B
Architecture	Dense, 8B	MoE, 35B/3B
Training data	Teacher (480B)	Self-mined
Training method	SFT	RS-SFT
Benchmarks	5	14 (+2 external)
Avg. improvement	+28.3%	+21.7pp
Teacher required	Yes (GPT-5)	No

6.6. Comparison with Zhang et al. (2026)

Our work differs from the original RLM paper (Zhang et al., 2026) in four key ways, summarized in Table 6.

The original paper achieves a larger average improvement (+28.3% vs our +21.7pp), but uses a 480B teacher to generate gold trajectories and evaluates on fewer benchmarks. Our *teacherless* approach—mining correct trajectories from the base model's own outputs—is entirely self-contained and reproducible without access to proprietary models. The quality of self-mined data is necessarily lower than teacher-generated data (the base model solves fewer tasks than GPT-5), but iterative mining partially compensates: each RS-SFT round produces a better model that mines harder trajectories for the next round. Our evaluation across 14 diverse benchmarks (vs. the original 5) provides a more comprehensive assessment of training quality, including tasks ($O(N^2)$ cross-document, code debugging) where trained models show minimal or no improvement—failure modes that a narrower evaluation might miss.

6.7. Key Insights Across Model Scales

1.7B vs 35B. At 1.7B (Appendix F), the model can barely write REPL code, requiring SFT warmup before RL. At 35B, the base model generates competent RLM code out of the box, enabling both direct RL and direct RS-SFT from base. This validates the choice of a larger model for RLM training.

LoRA creates planning-perception entanglement. The single most impactful discovery from the GRPO phase is that LoRA fine-tuning of recursive models creates an unintended coupling between planning (root code generation) and perception (sub-call QA). The hybrid architecture (Section 3.3) decouples these by routing sub-calls to the unadapted base model. RS-SFT partially mitigates this because it only trains on trajectories where both root and sub-call code produce correct results.

Implementation correctness is non-negotiable. Seven

bugs in the GRPO pipeline—wrong loss function (unclipped IS vs PPO), sub-query temperature leak, advantage normalization mismatch, and others—collectively prevented learning (loss values of $O(10^7)$). Only after fixing all seven did training produce stable improvements. Section F documents each bug and its fix, providing a practical reference for RL practitioners working with code-generation models.

6.8. When Does RL Fail for Code Generation?

Our extensive RL development (11 iterations, ∼84 API-hours) reveals a structural limitation of policy gradient methods for *multi-task code generation*: code has a narrow mode space where negative gradients are disproportionately destructive.

In natural language generation, suppressing one phrasing ("The answer is X") has minimal impact on other valid phrasings ("X is the answer," "The result is X"). The output distribution is broad enough that negative gradients can remove poor patterns without collateral damage. In code generation, the output distribution is much more concentrated—there are far fewer "correct" ways to write a chunk-and-search loop than ways to phrase an answer. Suppressing one code pattern (e.g., loose text parsing via `split("\n")`) often suppresses the *only* pattern the model knows for that operation, causing regressions across all tasks that use it.

This analysis suggests that **RL for code generation should either (a) use only positive signal** (RS-SFT, best-of-N), **(b) carefully partition the action space** so negative gradients for one task type cannot affect another (e.g., task-conditional adapters), or **(c) use process reward models** that provide per-turn credit assignment dense enough to avoid the cross-task interference caused by sparse outcome rewards. Our SC-GRPO addresses (b) partially via strategy conditioning, but the shared LoRA adapter still allows cross-task interference at the parameter level.

6.9. Limitations

Benchmark scale. Our benchmarks use 10–20 tasks per benchmark, limiting statistical power. The largest gains (Verbatim Copy +60pp, KV Retrieval +56pp) are robust, but moderate gains (±5pp) may not be statistically significant at these sample sizes.

Synthetic benchmarks. 14 of 16 benchmarks use synthetic data. Performance on real-world long-context tasks may differ, as evidenced by the OOLONG regression.

Single base model. We evaluate only Qwen3.5-35B-A3B. Results may differ with other base models or model sizes.

RS-SFT ceiling. RS-SFT can only learn what the base model (or previous RS-SFT iterations) already does well. For tasks where the base model has <10% success rate, trajectory mining produces insufficient training data. Teacher distillation from a larger model could address this.

Multi-NIAH regression. The one consistent regression (−4.3pp) on Multi-NIAH suggests that training on diverse task types may slightly reduce the model's ability to perform exhaustive search. This could potentially be addressed by upweighting Multi-NIAH in the training data, but we did not investigate this to avoid overfitting to the evaluation set.

7. Conclusion

We trained the first open-weight natively recursive language model based on Qwen3.5-35B-A3B, systematically comparing GRPO and RS-SFT training paradigms across 14 diverse long-context benchmarks. Our main findings:

1. **RS-SFT outperforms GRPO by +10.4pp** (61.6% vs 61.1% average). Training only on correct trajectories avoids the cross-task interference caused by GRPO's negative gradients, which suppress useful base-model patterns for tasks not in the training distribution. This finding challenges the assumption that RL should outperform behavioral cloning for code generation.
2. **GRPO on top of RS-SFT actively hurts.** Applying RL refinement to an RS-SFT checkpoint degrades 5 of 9 evaluated benchmarks. The SFT→RL pipeline is not universally beneficial.
3. **Format rigidity is the root cause of RL regressions.** GRPO trains rigid format-specific parsing in sub-call code that fails on output variation. RS-SFT avoids this through quality filtering for format-robust code.
4. **Iterative trajectory mining creates a data flywheel.** Each RS-SFT round produces a better model that mines better trajectories: V16 (70.4%) → V17 (71.6%).
5. **SFT scaling has diminishing returns.** V18–V21 (4 experiments with 1,732–4,660 samples) all score below V17. An oracle analysis reveals a 77.7% ceiling achievable via per-benchmark routing.
6. **SC-GRPO eliminates mode collapse** in code-generation GRPO (0% group collapse vs 60% in standard GRPO).

Future directions. (a) Per-benchmark model routing to approach the 77.7% oracle ceiling; (b) teacher distillation from Qwen3.5-397B-A17B for tasks where the base model has <10% success rate; (c) multi-level recursion where RLMs call RLMs; (d) larger evaluation suites ($N \geq 100$) to establish statistical significance; (e) investigation of whether DPO (which uses negative signal but is more stable than GRPO) can improve on RS-SFT.

Our results demonstrate that **rejection sampling SFT is the superior training paradigm for recursive language models**, achieving +21.7pp improvement over the base model across 14 benchmarks while avoiding the regressions that plague RL training. Notably, this is achieved *without any teacher model*—unlike the original RLM work (Zhang et al., 2026), which relies on a 480B teacher for trajectory generation, our iterative self-mining approach is fully self-contained. The combination of iterative trajectory mining, quality filtering, and task-balanced SFT provides a simple, stable, and effective recipe for training RLMs that is reproducible by anyone with access to the base model. All weights, code, benchmarks, and training data are publicly available.

Acknowledgements

This project was completed for CS234: Reinforcement Learning, Winter 2026 at Stanford University. Initial Qwen3-1.7B experiments used approximately 11.1 GPU-hours on 8×H200; the Qwen3.5-35B-A3B experiments used the Tinker remote training API. Everything is available at https://github.com/pythonomar22/rl4rlm (branch: `paper-and-release-updates`), including all source code, training code, and evaluation benchmarks. The trained LoRA adapters are available on HuggingFace: `omar81939/rlm-qwen35-35b-a3b`.

A. Trajectory Examples: Base vs. V10-s40 vs. V17

We present representative trajectories showing how training transforms model behavior. All examples are from seed 10000 evaluation.

A.1. Example 1: Hard Multi-Hop QA — Two-Phase Decomposition

Task: "Find the city where the Lead of Legal is based" (2-hop reasoning, 150K character document).

Base model (incorrect, returns "Dublin"):

```
chunk_size = 20000
overlap = 2000
results = []
i = 0
while i < len(context):
    chunk = context[i:i+chunk_size]
    answer = llm_query(
        f"Find the city where the Lead "
        f"of Legal is based. Return ONLY "
        f"the city name. If not found, "
        f"say 'NOT FOUND'.\n\n{chunk}")
    if "not found" not in answer.lower():
        results.append(answer.strip())
    i += chunk_size - overlap

from collections import Counter
result = Counter(results).most_common(1)[0][0]
FINAL_VAR("result")
```

The base model asks the *compound* question directly: "Find the city where the Lead of Legal is based." No single chunk contains both the person's identity and their city, so sub-calls return contradictory answers and the model picks the wrong one via majority voting.

V17 RS-SFT (correct, returns "Boston"):

```
# Step 1: Find who is the Lead of Legal
chunk_size = 20000
overlap = 2000
candidates = []
i = 0
while i < len(context):
    chunk = context[i:i+chunk_size]
    answer = llm_query(
        f"Who is the Lead of Legal? "
        f"Return ONLY the person's full "
        f"name. If not found, say "
        f"'NOT FOUND'.\n\n{chunk}")
    if "not found" not in answer.lower():
        candidates.append(answer.strip())
    i += chunk_size - overlap

from collections import Counter
lead_name = Counter(candidates) \
    .most_common(1)[0][0]
print(f"Found Lead of Legal: {lead_name}")


# Step 2: Find where this person is based
results = []
i = 0
while i < len(context):
    chunk = context[i:i+chunk_size]
    answer = llm_query(
        f"In which city is {lead_name} "
        f"based? Return ONLY the city "
        f"name. If not found, say "
        f"'NOT FOUND'.\n\n{chunk}")
    if "not found" not in answer.lower():
        results.append(answer.strip())
    i += chunk_size - overlap

final_city = Counter(results) \
    .most_common(1)[0][0]
FINAL_VAR("final_city")
```

V17 breaks the problem into two explicit steps: first find the person ("Rachel Thomas"), then find their city ("Boston"). Each atomic query is answerable from a single chunk. This two-phase decomposition is the hallmark of successful multi-hop reasoning in RLMs. **Hard Multi-Hop accuracy: Base 30% → V17 60% (+30pp).**

A.2. Example 2: Event Counting — Extract Then Count in Python

Task: "How many events were performed by Jack Anderson?" (150K character event log document).

Base model (incorrect, returns "7", expected "8"):

The base model asks sub-calls to *count* events directly: "How many events did Jack Anderson perform in this text?" Sub-calls hallucinate counts (returning "3" or "2" for chunks containing 4 events), and the base model sums these hallucinated counts.

V17 RS-SFT (correct, returns "8"):

```
chunk_size = 20000
overlap = 2000
events = []
i = 0
while i < len(context):
    chunk = context[i:i+chunk_size]
    answer = llm_query(
        f"Find ALL events performed by "
        f"Jack Anderson. Return ONLY the "
        f"raw event lines. If none found,"
        f" say 'NONE'.\n\n{chunk}")
    if "none" not in answer.lower():
        for line in answer.strip() \
                .split("\n"):
            line = line.strip()
            if line and "jack anderson" \
                    in line.lower():
                events.append(line)
    i += chunk_size - overlap

# Deduplicate and count in Python
unique_events = set(events)
count = len(unique_events)
FINAL(str(count))
```

V17 learns the critical pattern: **extract raw event lines, then count in Python**. Sub-calls are asked to return raw data (not counts), and the Python code handles deduplication and aggregation. This is the correct strategy for event counting—delegating counting to sub-calls produces hallucinated numbers because LLMs cannot reliably count items in text. **Event Counting accuracy: Base 46.4% → V17 73.0% (+26.6pp).**

A.3. Example 3: Key-Value Retrieval — Validation Strategy

Task: "Find registry entry ID 'ENG-14777' and return its numeric VALUE" (185K character registry document with 2,500+ entries).

V17 RS-SFT (correct, returns "6932.36"):

```
target_id = "ENG-14777"
chunk_size = 8000
overlap = 2000
results = []
i = 0
while i < len(context):
    chunk = context[i:i+chunk_size]
    answer = llm_query(
        f"Find registry entry with ID "
        f"'{target_id}'. Return ONLY the "
        f"VALUE (numeric). If not found, "
        f"say 'NOT FOUND'.\n\n{chunk}")
    if "not found" not in answer.lower():
        val = answer.strip()
        # Validate: must look numeric
        if val.replace(".", "") \
                .replace("-", "") \
                .isdigit():
            results.append(val)
    i += chunk_size - overlap

from collections import Counter
if results:
    result = Counter(results) \
        .most_common(1)[0][0]
else:
    result = "NOT FOUND"
FINAL_VAR("result")
```

V17 uses three key techniques: (1) smaller chunks (8K instead of 20K) for precise lookups in dense structured data; (2) **numeric validation** of sub-call responses before aggregation, preventing false positives from malformed answers; (3) majority voting across chunks for robustness. **KV Retrieval accuracy: Base 29.2% → V17 85.4% (+56.2pp).**

A.4. Example 4: Verbatim Copy — LLM-Based Extraction

Task: "Find and reproduce EXACTLY the paragraph containing 'CASE-2026-49099'" (10K character document).

V10-s40 GRPO (incorrect, returns the instruction text):

```
chunk_size = 20000
if "CASE-2026-49099" in context:
    pos = context.find("CASE-2026-49099")
    start = context.rfind("\n", 0, pos)
    end = context.find("\n", pos)
    paragraph = context[start+1:end]
    FINAL(paragraph.strip())
```

The GRPO model attempts direct string manipulation to find paragraph boundaries. This fails because the "paragraph" found by `rfind/find` on newlines captures the wrong text boundaries (in this case, the instruction text rather than the paragraph).

V17 RS-SFT (correct, score 1.0):

```
chunk_size = 20000
overlap = 2000
results = []
i = 0
while i < len(context):
    chunk = context[i:i+chunk_size]
```

```
    answer = llm_query(
        f"Find and reproduce EXACTLY the "
        f"paragraph containing "
        f"'CASE-2026-49099'. Copy the "
        f"entire paragraph verbatim. "
        f"Return ONLY the paragraph. If "
        f"not found, say 'NOT FOUND'."
        f"\n\n{chunk}")
    if "not found" not in answer.lower():
        results.append(answer.strip())
    i += chunk_size - overlap
from collections import Counter
result = Counter(results) \
    .most_common(1)[0][0]
FINAL_VAR("result")
```

V17 **delegates verbatim extraction to the LLM** via sub-calls, rather than attempting string manipulation. The sub-call model understands natural language paragraph boundaries and can reproduce text verbatim, which is more reliable than regex/string-based paragraph detection. **Verbatim Copy accuracy: Base 20% → V10-s40 60% → V17 80% (+60pp from base).**

A.5. Example 5: Multi-NIAH Regression Analysis

Task: Find 5 secret codes in a 50K character document.

Base model (100% recall, 5/5 found): Uses systematic chunking with careful aggregation. At 99.4% average recall, the base model is already near-perfect on this benchmark.

V17 RS-SFT (60% recall, 3/5 found): V17's code pattern is similar but misses 2 codes at chunk boundaries. The training data was biased toward "find some items" tasks (multi-hop, event counting) rather than "find *all* items" tasks.

This is V17's only regression (−4.3pp). The regression is small (95.1% vs 99.4%), and the base model's 99.4% may benefit from chance—at n=20 tasks, this difference is within statistical noise. Nevertheless, the pattern suggests a genuine tradeoff: training on diverse task types slightly reduces exhaustive-search capability.

B. Full Per-Version Comparison

Table 7 shows the complete results for all major model versions across all 14 benchmarks.

Key observations from the full comparison:

1. **V16 RS-SFT already beats V10-s40 GRPO** on 10 of 14 benchmarks. The jump from 61.1% to 70.4% (+9.3pp) is the single largest improvement from any training method change.

2. **V16's strengths are different from V10-s40's.** V16 excels on DataFrame QA (99.0% vs 85.0%), Hard Multi-Hop (70.0% vs 30.0%), and Verbatim Copy (100.0% vs 60.0%), while V10-s40 is better on NIAH (75.0% vs 65.0%) and OOLONG (20.0% vs 20.0%).

3. **V17 improves over V16 on retrieval tasks.** The Round 2 mining from V16 adds trajectories for tasks that V16 solved but the base model couldn't, producing large gains on KV Retrieval (59.2% → 85.4%) and Event Counting (61.6% → 73.0%).

4. **No single model dominates all benchmarks.** The best model per benchmark is spread across V16, V17, V18, V19, and V20, confirming the benchmark tradeoff finding.

5. **V16+GRPO is uniformly worse than V16 RS-SFT** on every benchmark where both were evaluated, confirming that GRPO refinement damages RS-SFT checkpoints.

C. Format Rigidity: Detailed Analysis

We analyze 50 trajectories from V10-s40 (GRPO) and V17 (RS-SFT) on cross-document comparison and DataFrame QA tasks—the two benchmark types most affected by format rigidity.

C.1. Pattern 1: Strict Format Parsing

GRPO-trained V10-s40 generates code that assumes specific sub-call output formats:

```
# V10-s40: Expects "ORG: Name" format
for line in answer.split("\n"):
    if "ORG: " in line:
        parts = line.split("ORG: ")
        org_name = parts[1].strip()
        orgs.append(org_name)
```

When the sub-call returns "Organization: Acme Corp" or "The organization is Acme Corp", this code silently fails (no match for "ORG: ").

V17 RS-SFT generates format-agnostic code:

```
# V17: Accepts any format
for line in answer.strip().split("\n"):
    line = line.strip()
    if line and line not in seen:
        orgs.append(line)
        seen.add(line)
```

V17's code accepts any line-separated output, making it robust to sub-call format variation.

C.2. Pattern 2: Single-Template Convergence

GRPO training converges to a single code template for each task type. Across 20 DataFrame QA trajectories from V10-s40, 85% use identical chunk sizes (20000) and identical sub-call prompts. In contrast, V17 trajectories show more

Table 7. Complete per-benchmark results across all major model versions (seed 10000, 14 benchmarks). Best per-benchmark in **bold**. V10-s40 is the best GRPO model; V17 is the best RS-SFT model; V18–V21 are SFT scaling experiments.

Benchmark	Base	V10-s40	V16-SFT	V16+GRPO	V17	V18	V19	V20
NIAH	65.0	75.0	65.0	65.0	75.0	**80.0**	70.0	70.0
Multi-NIAH	**99.4**	90.0	95.0	96.1	95.1	93.4	96.8	95.0
Hard NIAH	83.3	93.3	83.3	83.3	**96.7**	90.0	93.3	83.3
Doc-Classify	56.3	76.6	76.7	83.6	80.2	**87.9**	78.0	84.7
DataFrame QA	75.0	85.0	**99.0**	94.0	93.0	94.0	97.0	94.0
Code Debug	50.0	43.3	50.0	35.6	**60.0**	40.0	46.7	43.3
Multi-Hop QA	55.0	60.0	75.0	75.0	**80.0**	70.0	70.0	45.0
Hard Multi-Hop	30.0	30.0	**70.0**	—	60.0	40.0	40.0	50.0
Notebook QA	46.7	63.3	83.3	73.3	70.0	76.7	83.3	**90.0**
Event Counting	46.4	41.7	61.6	—	**73.0**	61.6	57.2	67.0
Cross-Doc	42.2	42.9	**47.0**	—	43.4	**47.0**	34.7	45.4
KV Retrieval	29.2	75.0	59.2	—	85.4	75.0	86.7	**88.9**
Verbatim Copy	20.0	60.0	**100.0**	80.0	80.0	90.0	**100.0**	**100.0**
OOLONG	0.0	20.0	20.0	—	10.0	**30.0**	20.0	15.0
Avg (14)	49.9	61.1	70.4	—	**71.6**	69.7	69.5	69.4
W/L/T vs Base	—	9/3/2	12/2/0	—	**13/1/0**	12/2/0	11/3/0	10/4/0

variation: chunk sizes range from 8000 to 25000, and sub-call prompts vary based on the specific question type.

C.3. Pattern 3: Error Recovery

When sub-calls return unexpected output, V17 trajectories more frequently include error handling and retry logic (in multi-turn execution). V10-s40 trajectories tend to fail silently and return incorrect aggregations without detection.

D. RS-SFT Data Pipeline Details

D.1. Trajectory Mining

The mining script (`scripts/mine_eval_trajectories.py`) scans all evaluation result directories for correct trajectories. For each `eval_results.json` file:

1. Load per-task results, including the full trajectory (turns, code, outputs)
2. Check the appropriate score field (`score` for most benchmarks, `accuracy` for doc-classify, `recall` for multi-NIAH)
3. Filter trajectories with score ≥ 0.9
4. Extract the conversation format (system prompt + assistant turns with code)
5. Apply content-based deduplication: key = task_type + hash(completion text)

Bug discovery: V10–V17 training data contained **zero** mined doc-classify and multi-NIAH trajectories because the mining script only checked the `score` field. Doc-classify uses `accuracy` and multi-NIAH uses `recall`. This was fixed in V18, adding 858 doc-classify and 798 multi-NIAH trajectories. Despite this fix, V18 (69.7%) underperformed V17 (71.6%), suggesting the additional data diluted rather than complemented V17's training distribution.

D.2. Quality Filtering

The filtering pipeline (`scripts/filter_trajectories.py`) applies 8 quality criteria (Section 3.4). Key statistics for V17:

- Raw mined trajectories: ∼8,200
- After score filter (≥0.9): ∼5,800
- After deduplication: ∼4,900
- After quality filters: ∼4,100
- After task balancing (max 200/type): 3,644

D.3. Task Balancing

To prevent any benchmark from dominating the loss, we cap each task type at 200 samples (selecting the top 50% by score + random sampling from the remainder). Task types with fewer than 30 samples are oversampled. The final V17 training set has 3,644 samples balanced across 14 benchmark types.

E. Training Dynamics

E.1. GRPO Training Dynamics

V4-s5 (Standard GRPO, 5 steps from base). V4-s5 is the initial GRPO run at 35B scale, using the debugged pipeline

from the 1.7B experiments. Loss values are well-behaved: $107, -62, 497, -35, -70$ (order $O(10^2)$, confirming PPO clipping works). The model achieves +0.5pp average improvement in 5 steps, primarily on search tasks (NIAH +10pp, Doc-Classify +17.6pp).

V7 (Importance Sampling Bug, 5 steps). V7 switches back to unclipped importance sampling (`"importance_sampling"` instead of `"ppo"`) while increasing gradient accumulation. Losses explode: 4,336,545, −933,989, 11,516,850—**100,000**× larger than V4. The root cause: with 48–55 datums accumulated per step, the policy drifts significantly within each step. By the time later datums are processed, $\rho_t = \pi_\theta/\pi_{\text{old}}$ has diverged dramatically because π_θ has already been updated by earlier datums in the same batch. PPO clipping bounds this ratio; unclipped IS does not.

V10-s40 (SC-GRPO, 40 steps from V4-s5). The best GRPO model trains with SC-GRPO and regression-targeted task distribution. Reward trajectory: 0.42 (step 0) → 0.62 (step 40), with 3 mode collapse events (steps where all K=8 trajectories produce identical code, yielding zero advantages and no gradient). The regression-targeted distribution allocates 70% of training weight to previously-regressing tasks, reducing oscillation amplitude compared to uniform-task GRPO.

Mode collapse events in V10-s40. Despite SC-GRPO, 3 of 40 steps (7.5%) still experience full group collapse. These occur exclusively on DataFrame QA tasks, where the CSV parsing pattern is so stereotyped that even different strategy prompts produce identical code. On average, SC-GRPO reduces collapse from 60% (standard GRPO) to <10% of groups, but cannot eliminate it entirely for highly deterministic task types.

E.2. RS-SFT Training Dynamics

V17 SFT training converges smoothly: cross-entropy loss decreases from ∼2.8 to ∼0.9 over 3 epochs (∼900 steps). No instability, no collapse events, no reward oscillation—a stark contrast to GRPO's training dynamics. Training time: 2.8 hours on Tinker API.

Stability comparison. Over equivalent training budgets: GRPO experiences 3–5 collapse events, reward oscillation amplitude of ±0.15, and 7.5% of steps with zero gradient. RS-SFT experiences none of these pathologies. This stability advantage compounds over training—GRPO must "waste" collapsed steps, while RS-SFT extracts value from every step. The practical implication: RS-SFT requires minimal hyperparameter tuning, while GRPO requires careful calibration of temperature schedules, KL coefficients, advantage normalization, and task distributions.

E.3. V18–V21: Diminishing Returns in Detail

Each post-V17 experiment shows the same pattern: rapid loss decrease during training, followed by evaluation revealing unexpected benchmark regressions. The most dramatic example is V21 (targeted fine-tuning at lr 3×10^{-6} for only 108 steps), which still causes cross-doc to collapse from 43.4% to 18.5%. This extreme sensitivity suggests that V17's learned representations for cross-document reasoning occupy a narrow basin of the loss landscape, easily disrupted by even gentle gradient updates.

F. Qwen3-1.7B RL Development: A Complete Training Pipeline

We developed the full RL training pipeline on Qwen3-1.7B (local 8×H200 cluster) before scaling to Qwen3.5-35B-A3B via the Tinker API. This section documents the complete RL development journey: 4 GRPO versions, each fixing a fundamental issue discovered in the previous, plus DPO and STaR experiments. These 1.7B experiments are the foundation of every RL design decision in the 35B experiments.

F.1. Policy Initialization: SFT and STaR

At 1.7B parameters, the base model can barely write valid REPL code. **SFT serves as behavioral cloning** to initialize the policy before RL (Ouyang et al., 2022).

SFT-v1 failure (68% → 56%). Including error turns in the SFT data taught the model to reproduce error patterns (e.g., calling undefined variables). The policy learned to imitate failure modes alongside successes—a classic behavioral cloning pitfall.

SFT-v2 success (68% → 90% NIAH). Strict filtering to only clean, correct turns dramatically improved performance. Long-document accuracy jumped: 50K from 55% to 80%, 100K from 60% to 80%. This validated that behavioral cloning from high-quality demonstrations is effective when the data is pure—foreshadowing our later RS-SFT finding at 35B scale.

SFT-v2 data balance failure. Despite SFT-v2's headline success, 114 of 155 training samples (73.5%) were NIAH. This caused **catastrophic task collapse**: Multi-NIAH recall dropped from 97.8% to 0% because the model learned to return a single answer (NIAH pattern) instead of collecting multiple answers. Lesson: task-balanced batching is essential for multi-task training.

STaR (Zelikman et al., 2022). The SFT model generates trajectories on harder tasks (multi-needle, doc-classify), correct ones are filtered and added to the training set, and a second SFT round extends capability to new task types. This iterative self-improvement loop is the precursor to our later

iterative trajectory mining (Section 3.4).

F.2. GRPO v1–v4: Systematic RL Debugging

Each GRPO version fixes a fundamental issue discovered in the previous version, providing a case study in the engineering challenges of applying policy gradient methods to code generation.

GRPO-v1: Reward collapse and mode collapse. We train with K=4 trajectories, binary reward (1.0 if correct, 0.0 otherwise), and no KL regularization. Within 10 steps, the model achieves 100% accuracy on all training tasks. The advantages become uniformly zero ($\hat{A}_i = 1.0 - 1.0 = 0$), producing zero gradient—reward collapse. Simultaneously, all K=4 trajectories converge to identical code by step 18. The training dynamics show the collapse clearly:

Steps	Updates/step	Reward	Status
1–10	15–24	0.54 → 0.84	Learning
11–17	8–16	0.84	Converging
18–30	**0**	0.84	**Stagnated**

Fix: Switch from binary to continuous rewards (e.g., multi-needle recall $\in [0, 1]$) to maintain non-zero advantages even when all trajectories succeed. Increase K=4 → K=8 for more advantage diversity.

GRPO-v2: Training oscillation without KL. Continuous rewards from multi-needle recall provide gradient signal, but training oscillates wildly without KL regularization. The reward trajectory shows: $0.73 \rightarrow 0.21 \rightarrow 0.76 \rightarrow 0.38 \rightarrow 0.81$. The policy overshoots on each update, moving far from the previous policy, then overcorrects. Oscillation amplitude: 0.548 (range $[0.21, 0.76]$).

Fix: Add KL regularization to prevent large policy updates. But which form of KL?

GRPO-v3: Weight-space KL failure. We implement weight-space L2 as a KL proxy: $\mathcal{L}_{\text{KL}} = \beta \cdot \|\theta - \theta_{\text{ref}}\|^2/d$, where d = 17.4M (LoRA parameter count). The per-parameter L2 values are $O(10^{-10})$—effectively zero regularization even at β=0.02. The policy drifts freely, causing **catastrophic regression**: Multi-NIAH collapses from 97.8% to **41.4%** in 10 steps. This is the most severe policy collapse we observed.

Root cause: LoRA's low-rank structure means small weight changes can produce large distributional shifts. A perturbation of 10^{-5} per parameter across 17.4M parameters can significantly alter the output distribution, but weight-space L2 registers this as negligible.

Fix: Replace weight-space L2 with *token-level* KL divergence via a frozen reference model. Compute KL = $\frac{1}{T}\sum_{t=1}^{T} \log \pi_{\text{ref}}(y_t|y_{<t}) - \log \pi_\theta(y_t|y_{<t})$ and subtract $\beta_{\text{KL}} \cdot$ KL from the reward. This produces meaningful regularization: KL grows smoothly from 0.0008 to 0.0197 over 30 steps, providing a direct measure of policy drift.

GRPO-v4: Stable training with conditioned log-probabilities. The combination of token-level KL and *conditioned* log-probabilities (computing $\pi_\theta(y|x)$ rather than unconditional $\pi_\theta(y)$) finally produces stable training. Oscillation amplitude halves compared to v2 (0.26 vs 0.55). Multi-NIAH recall improves from 38.5% (base) to 85.3% (GRPO-v4). The training reward trajectory is monotonically improving for the first 20 steps before plateauing.

The conditioned logprob insight. GRPO-v3 and v4 use identical algorithms, hyperparameters, and data. The *only* difference is conditioned vs. unconditional log-probabilities. V3 produces 41% Multi-NIAH; v4 produces 85%. Unconditional logprobs measure $P(y)$—the probability of the code regardless of the prompt. Conditioned logprobs measure $P(y|x)$—the probability of this code *given this specific task*. Training on unconditional logprobs teaches the model to produce "likely code in general," not "good code for this task."

F.3. DPO: Implicit Reward, Explicit Stability

DPO (Rafailov et al., 2023) replaces the explicit reward model with an implicit one derived from preference pairs. We construct 25 preference pairs from GRPO-v4's trajectories: each pair shares a prompt, with the higher-scoring trajectory as "chosen" and the lower-scoring as "rejected."

Training produces surprisingly small loss changes ($0.693 \rightarrow 0.684$ over 30 steps) but **large downstream effects**: Multi-NIAH recall jumps by +29.5pp. DPO is more stable than GRPO (no oscillation, no collapse events) but still uses negative signal—the gradient pushes the policy *away* from rejected trajectories, which can cause the same format rigidity issues we later observe at 35B scale.

Comparison with GRPO-v4: DPO achieves stronger Multi-NIAH recall (100% at 3 needles, 87.5% at 8 needles) than GRPO-v4 (100% at 3, 57.8% at 8). However, DPO slightly regresses on NIAH at 100K characters (78% vs GRPO-v4's 72%—both below SFT's 80%). Both methods show the same length-vs-breadth tradeoff observed later at 35B scale.

F.4. Key RL Lessons from 1.7B

These experiments yield five RL-specific insights that directly inform our 35B pipeline:

1. **Reward design determines learnability.** Binary rewards cause reward collapse; continuous rewards enable learning. For multi-task training, per-task reward functions with auxiliary bonuses (format, persistence, sub-calls) are essential.

Table 8. NIAH accuracy by document length (characters), Qwen3-1.7B. Each cell: 20 tasks.

MODEL	5K	10K	20K	50K	100K
BASE	85	80	80	55	60
SFT	**100**	95	95	80	80
RL-V2	**100**	**100**	95	80	85
STAR	75	90	85	90	95
RL-V3	80	95	85	90	**100**
DPO	**100**	80	90	**95**	78
GRPO-V4	**100**	90	90	91	72

2. **Conditioned log-probabilities are essential.** GRPO-v3 and v4 differ only in this choice, yet produce opposite results (41% vs 85%). This is not documented in the GRPO paper (DeepSeek-AI, 2025) and may be specific to code generation where unconditional code probability is dominated by boilerplate patterns.

3. **Weight-space KL is inadequate for LoRA.** Small LoRA weight perturbations produce large distributional shifts that weight-space L2 cannot detect. Token-level KL via a frozen reference forward pass is necessary.

4. **Data balance prevents task collapse.** Behavioral cloning (SFT) on imbalanced data causes catastrophic forgetting of minority tasks. Task-balanced batching—which we carry forward to RS-SFT—is essential.

5. **Implementation bugs compound catastrophically.** Seven bugs in the RL pipeline—wrong loss function, sub-query temperature leak, advantage normalization, and others—collectively produced loss values of $O(10^7)$. Each bug seemed minor in isolation; together they prevented any learning. Systematic debugging via careful ablation (changing one component at a time between v1→v2→v3→v4) was the only path to a working pipeline.

Transition to 35B. Armed with a debugged RL pipeline and five hard-won insights, we scale to Qwen3.5-35B-A3B. At 35B, the base model writes competent REPL code without SFT warmup, enabling direct RL training from the base policy. The 1.7B bugs are fixed, the reward function is tuned, and KL regularization is validated—all before spending any compute on the target model.

G. NIAH Breakdown by Document Length (Qwen3-1.7B)

Table 8 shows NIAH accuracy stratified by document length for the 1.7B experiments.

SFT improves long-document performance (50K: 55% → 80%, 100K: 60% → 80%). RL-v3 achieves 100% on 100K but regresses on 5K (80% vs SFT's 100%). DPO and GRPO-v4 achieve 100% on 5K but regress at 100K (78% and 72%), suggesting a trade-off influenced by the length distribution in training data.

Table 9. Multi-NIAH recall by number of needles, Qwen3-1.7B. Each cell: 4 tasks.

MODEL	3	5	8	10
BASE	50.0	45.0	31.3	27.5
SFT	75.0	47.5	56.3	65.0
RL-V2	50.0	67.5	37.5	67.5
STAR	50.0	70.0	51.6	57.5
RL-V3	50.0	57.5	26.6	30.0
DPO	**100.0**	87.5	**81.3**	90.0
GRPO-V4	**100.0**	**97.5**	57.8	**100.0**

Table 10. Document classification accuracy by number of documents, Qwen3-1.7B. Each cell: 5 tasks.

MODEL	5	10	15	20
BASE	68.0	78.0	85.3	90.0
SFT	80.0	78.0	82.7	89.0
RL-V2	**88.0**	66.0	82.7	89.0
STAR	84.0	78.0	82.7	89.0
RL-V3	84.0	78.0	82.7	**91.0**
DPO	84.0	76.0	81.3	89.0
GRPO-V4	80.0	78.0	84.0	**91.0**

H. Multi-Needle Retrieval Breakdown (Qwen3-1.7B)

DPO and GRPO-v4 produce dramatic improvements over all prior methods. DPO shows consistently strong performance across all needle counts (81–100%). RL-v3 catastrophically fails on 8-needle (26.6%) and 10-needle (30.0%)—the most severe policy collapse observed at 1.7B scale.

I. Document Classification Breakdown (Qwen3-1.7B)

Document classification is the most stable benchmark: all trained models achieve 81–84% average with minimal variation. The scaffold structure already solves the $O(N)$ decomposition; training primarily affects sub-call classification quality.

Phase (Qwen3-1.7B)	Time	GPU-h
Trajectory collection (150 tasks)	38 min	0.63
SFT training (5 ep.)	34 sec	0.01
RL-v1 training (100 st.)	17 min	0.29
RL-v2 training (30 st.)	88 min	1.47
STaR collection (74 tasks)	30 min	0.50
STaR SFT (5 ep.)	32 sec	0.01
RL-v3 training (40 st.)	55 min	0.92
DPO (gen. + train)	92 min	1.53
GRPO-v4 (30 st.)	106 min	1.76
Evals ($\times$8 models)	$\sim$4 hr	$\sim$4.0
Total (1.7B)	$\sim$11 hr	**$\sim$11.1**

Phase (Qwen3.5-35B-A3B, Tinker API)	API-hours
Base model evals (14 benchmarks)	$\sim$3.0
V4-s5 GRPO (5 steps)	$\sim$2.5
V10-s40 SC-GRPO (40 steps)	$\sim$20.0
V10-s40 evaluation	$\sim$3.0
Trajectory mining (3 rounds)	$\sim$15.0
V16 RS-SFT training	$\sim$2.5
V17 RS-SFT training	$\sim$2.8
V16+GRPO experiment	$\sim$5.0
V18–V21 experiments	$\sim$10.0
All evaluations ($\times$8+ models)	$\sim$20.0
Total (35B)	**$\sim$84**

J. Computational Cost Breakdown

K. Strategy-Conditioned GRPO: Strategy Descriptions

The 10 strategy prompts used in SC-GRPO training. Each is prepended to the system prompt for that trajectory, forcing the model to adopt a different decomposition approach.

1. **Standard**: "Process the context by dividing it into chunks of approximately 15,000–20,000 characters each. For each chunk, use llm_query to extract relevant information. Aggregate results at the end."

2. **Extract-Compute**: "Focus on extracting raw data from the context first. Use llm_query to extract specific values, numbers, or entities from chunks. Then perform all computation (counting, sorting, comparing) in Python code rather than asking the LLM to compute."

3. **Binary Search**: "If you are looking for a specific piece of information, use a binary search approach: start with the middle of the context, determine which half likely contains the answer, and recurse on that half."

4. **Map-Reduce**: "Use a map-reduce pattern: map llm_query over all chunks to extract partial results, then reduce (merge, deduplicate, aggregate) the results in Python."

Table 11. Benchmark performance by complexity class. Δ is V17 vs Base.

Complexity	Base Avg	V17 Avg	Δ
$O(1)$: 4 benchmarks	49.4	84.0	+34.6
$O(K)$: 5 benchmarks	61.4	76.2	+14.8
$O(N)$: 3 benchmarks	59.2	82.1	+22.9
$O(N^2)$: 1 benchmark	42.2	43.4	+1.2
External: 1 benchmark	0.0	10.0	+10.0
All 14	**49.9**	**71.6**	**+21.7**

5. **Two-Pass**: "Use a two-pass approach: first pass scans the entire context to identify candidate locations or items, second pass verifies each candidate with focused queries."

6. **Small Chunks**: "Use small chunks of approximately 5,000 characters with 2,000-character overlap. This sacrifices speed for precision—smaller chunks mean the LLM sees more focused text and is less likely to miss details."

7. **Table Preserve**: "If the context contains structured data (CSV, tables, lists), preserve the structure when chunking. Do not split in the middle of a row or entry. Parse headers separately and include them with each chunk."

8. **Notebook Sequential**: "Process the context sequentially, tracking state as you go. For notebook-like content, process cells in order, maintaining a running summary of variables and results seen so far."

9. **Lookup Thorough**: "For lookup tasks, be thorough: use small chunks (5K–8K), search every chunk, and verify any found result by searching neighboring chunks for confirmation."

10. **Precision Extract**: "Focus on extracting exact text, not summaries. When using llm_query, always ask for exact quotes, exact values, or exact text. Never ask for summaries or paraphrases."

L. Benchmark Complexity Analysis

Table 11 classifies our benchmarks by algorithmic complexity and shows how training affects each class differently.

$O(1)$ **tasks benefit most from training** (+34.6pp average). These tasks require precise retrieval from a single location, and RS-SFT effectively teaches the model to search systematically with small chunks and validation.

$O(N^2)$ **tasks are hardest to improve** (+1.2pp). Cross-document comparison requires extracting information from two separate documents and comparing them—a complex

operation that current training data barely covers. This represents the most promising direction for future improvement.

References

Bai, Y., Tu, S., Zhang, J., Peng, H., Wang, X., Lv, X., Cao, S., Xu, J., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. *arXiv preprint arXiv:2412.15204*, 2024.

Bertsch, A., Pratapa, A., Mitamura, T., Neubig, G., and Gormley, M. R. OOLONG: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*, 2025a.

Bertsch, A., Pratapa, A., Mitamura, T., Neubig, G., and Gormley, M. R. OOLONG: Evaluating long context reasoning and aggregation capabilities. *arXiv preprint arXiv:2511.02817*, 2025b.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., and Ginsburg, B. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022.

Kamradt, G. Needle in a haystack — pressure testing LLMs. 2023.

Liu, Y. et al. Dr. GRPO: Removing bias from group relative policy optimization. *arXiv preprint arXiv:2503.20783*, 2025.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 2022.

Qwen Team. Qwen3.5 technical report. 2025.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Singh, A., Co-Reyes, J. D., Agarwal, R., Anand, A., Patil, P., Garcia, X., Liu, P. J., Harrison, J., Lee, J., Xu, K., Parisi, A., Kumar, A., Alemi, A., Rizkowsky, A., Nova, A., Adlam, B., et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2024.

Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., and Zhou, J. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2024.

Zelikman, E., Wu, Y., Mu, J., and Goodman, N. D. STaR: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35, 2022.

Zhang, A., Kraska, T., and Khattab, O. Recursive language models. *arXiv preprint arXiv:2512.24601v2*, 2026.